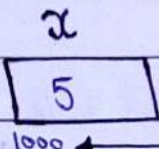


Pointers ÷

α
`int  $\alpha$  = 5;`

 address
reference

we can not change address

we also can not decide the address

`printf("%d",  $\alpha$ );` // 5
`printf("%d", & $\alpha$ );` // 1000

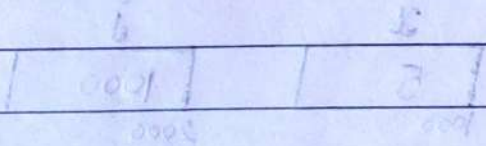
$\&$ → operator
 address of
 referencig operator
 unary operator

$*$
 Indirection operator
 Derefencig operator
 unary operator

variable → $\&$ → address

address → $*$ → variable

`printf("%d", * $\&\alpha$ );` // 5



int x = 5;

&x = 7;

↑ error

&x is just a way to represent address of variable x. &x is treated as a constant &x is not a variable.

constant = error

→ assignment ke left side constant hai toh its an error.

Pointer Variables ÷

A pointer is a variable that stores memory address.

Declaration of pointer ÷

data_type * pname; → informs the compiler that the variable is declared as a pointer.
→ name of the pointer variable

int x = 5;

int *j;

j = &x;

printf("%d %d %d", j, &x, x); // 1000 1000 5

printf("%d %d %d", *j, *&x, &j); // 5 5 2000

*&x = x
*j = x

in this question

Concept of Pointer

int x = 5

int *p;

p = &x;

int *q

X

int **q;

✓

q = &p;

Because q is not storing the address of an ordinary variable. It is storing address of an pointer named as p.

Pointer Arithmetic

int a, b, *p, *q;

p = &a;

q = &b;



p+q

p*q

p/q

p*5

p/4

It will give error

We can not do these arithmetic operation with two addresses.

p+2 { 1000 + 2*4 = 1008 }

p-3 { 1000 - 3*4 = 988 }

q-p = 2000 - 1000 = 1000 bit = ~~1000~~ = 250

Both should have same type.

Pointer is int type

Call by value $\div$ passes a copy of the value to the function.
So the changes made on the value doesn't affect the original value.

Call by reference $\div$ passes the actual value's memory location, allowing changes to the parameter to modify the original value.

Pointer and Array $\div$

- array to pointer notation :

$a[3]$
 $= *(a+3)$

$*(p+0) \approx a[0]$
 $*(p+1) \approx a[1]$
 $*(p+2) \approx a[2]$
 $*(p+i) = a[i]$

$a[3][2]$
 $\rightarrow a[3][2]$

$*(x+[2])$
 $*(a[3]+[2])$
 $*(*(a+3)+2)$

$p[3] = a[3]$
 $*(p+3) = *(a+3)$

$a[3][5][2]$
 $\rightarrow a[3][5][2]$

$*(x+[2])$
 $*(a[3][5]+[2])$
 $*(*(x+[5])+[2])$
 $*(*(a[3]+5)+2)$
 $*(** (a+3)+5)+2)$

Array of Pointer

```
int a[] = { 10, 20, 30, 40 };
    a[0] a[1] a[2] a[3]
    10   20   30   40
    1000 1004 1008 1012
```

```
int *p[4]; // *p[0], *p[1], *p[2], *p[3]
```

```
int i;
```

```
printf("Values at particular address = ");
```

```
for(i=0; i<4; i++)
```

```
{
```

```
    p[i] = &a[i]; // we store address of each element in
    printf("\n %d", *p[i]); // an Array
```

```
}
```

```
return 0;
```

• #include <stdio.h>

void main() → In C, the convention is to define the main function as int main

and to return 0 from it to indicate that the program executed successfully

int *ptr[] = { 11, 22, 33, 44 }; → It is not valid because initializing

int i = 1; an array of pointer with integer value is not correct.

while (i < 4) { It will run but it will warning }

```
{
```

```
    printf("%d\t", *(ptr + (i-1)));
    i++;
```

```
}
```

```
}
```

Output :

11 22 33 { with warning }

Correct way →

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int a[4] = {11, 22, 33, 44};
```

```
int *ptr[4] = {a, a+1, a+2, a+3};
```

```
int i = 1;
```

```
while (i < 4)
```

```
{
```

```
printf("%d\t", *(*(ptr + (i - 1))));
```

```
i++;
```

```
}
```

```
}
```

→ because here we define two array
one a array which has value and
another array ptr which is consisting
address of array a.

Output :-

11 22 33

$a[0] = 11$ $a[1] = 22$ $a[2] = 33$ $a[3] = 44$

$ptr[0] = a$ $ptr[1] = a + 1$ $ptr[2] = a + 2$ $ptr[3] = a + 3$

$ptr[0] = a$ $ptr[1] = a + 1$ $ptr[2] = a + 2$ $ptr[3] = a + 3$

$a[0] = 11$

$X : (9V * "80")$

(array of array)

Void Pointer

It is not having any associative data-type.

`int *ip;` Pointer to integer or It is storing an address of
`int *p;` variable whose data-type is int.
 ip is a pointer which is indicating to the int block.

`Float *fp;` Pointer to float

`char *cp;` Pointer to char

`void *vp;` Void pointer can store address of any variable.

`int a = 5;` `ip = &a` `ip = &b` X

`float b = 2.5;` `ip = &b`

`char c = 'd';` `ip = &c`

but void pointer can store address of any variable with any data-type.

- `vp = &a;`
`printf("%d", *vp);` X

→ We can't dereference the void pointer because compiler doesn't know the data-type of vp (void pointer).

So, for dereferencing void pointer we have to do type-casting.

```
void *vp;
```

```
vp = &a;
```

```
printf("%d", *(int*)vp);
```

└─ Type casting void pointer
into int

• #include <stdio.h>

```
void main()
```

```
{
```

```
void *vp;
```

```
int a = 5;
```

```
float b = 1.50;
```

```
char ch = 'c';
```

```
vp = &a;
```

```
printf("Value at a = %d", *(int*)vp);
```

```
}
```

└─ Type casting void pointer

Output

Value at a = 5

Null Pointer

```
#include <stdio.h>
```

```
int main ( )
```

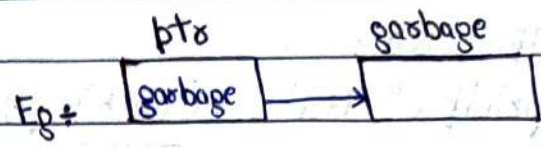
```
{
```

`int *ptr;` → we have not initialized the pointer

`printf ("%d", *ptr);` → so it will show undefined behaviour

```
}
```

- It might give some garbage value etc.



```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

`int *ptr = NULL or 0;`

Not Integer ←

Zero (0) is the corresponding value of NULL

It is itself a pointer

So, when a NULL or 0 is assigned to the pointer then it is considered as a null pointer.

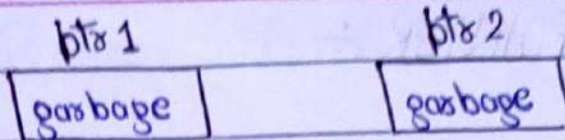
`printf ("%d", *ptr);` → dereferencing null pointer will crash the program

Because null pointer doesn't point to any valid object.

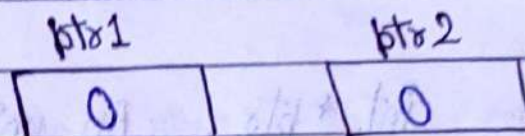
or

Null pointer points to 0th memory location which is reserved and we can't dereference that memory location.

* Concept



int * ptr 1; } It is not a guaranty that ptr 1 == ptr 2
int * ptr 2;



int * ptr 1 = NULL; } ptr 1 == ptr 2 always
int * ptr 2 = NULL;

WILD POINTER ÷

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int *ptr;    An uninitialized pointer act as wild pointer  
    printf("%d", *ptr);
```

↳ On dereferencing wild pointer it shows undefined behaviours such as -

- It can give some garbage value
- It can crash the program
- It create nasty bug in program

DANGLING POINTER ÷

```
#include <stdio.h>
```

```
int main()
```

```
{
```

ptr

100

10

100

X → memory is freed

int *ptr = (int *) malloc (size of (int)); → Asking malloc to allocate memory of size of int

*ptr = 10;

printf("%d", *ptr); // 10

free(ptr); It is used to free the memory

But still the pointer (*ptr) is pointing to the deallocated memory.

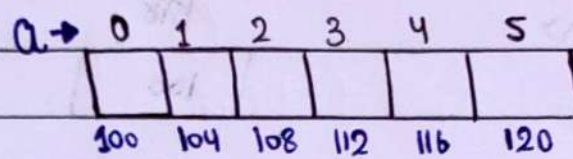
printf("%d", *ptr); Now if we try to dereference pointer it shows undefined behaviour

A pointer variable that holds the address of in-active area location then the pointer is called dangling pointer

So to overcome from situation created by WILD and DANGLING pointer it to the pointer variable as NULL.

Pointer Arithmetic

• Addition



int *p; p = &a;

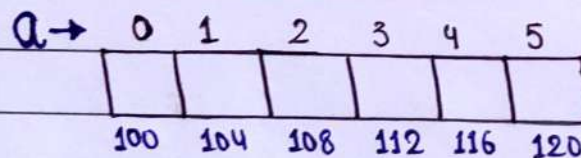
p = p + 3;

↳ Means moving the pointer 3 positions in forward direction.

Addition in pointer

$p = p + 3 \times \text{sizeof(int)}$

• Subtraction



int *p;

p = &a[5];

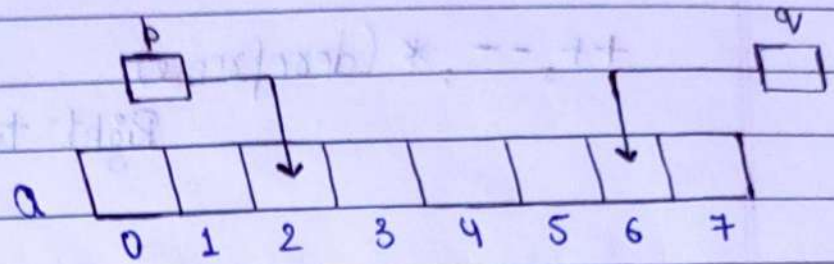
p = p - 4;

↳ Means moving the pointer 4 positions in backward direction.

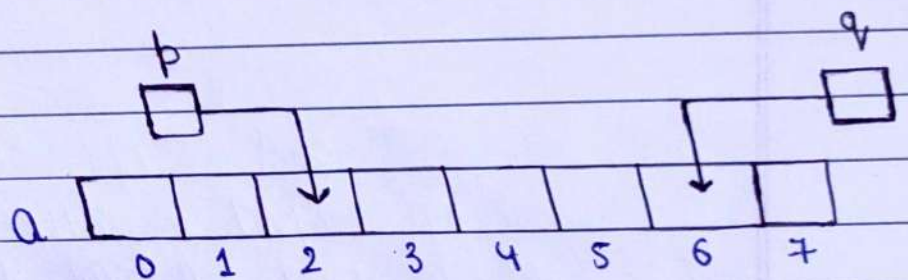
Subtraction in pointer

$p = p - 4 \times \text{sizeof(int)}$

Subtracting one pointer from another pointer +



$q - p = 4$ (pointer p pointing index 2 & pointer q pointing index 6)



If base address of array is 1000.

$$p = 1008$$

$$q = 1024$$

$$\Rightarrow \frac{q - p}{\text{sizeof(int)}}$$

address containing pointer q - address in p
sizeof(int)

↓
if pointer containing character then sizeof(char)